INDEX

120 PAGES

Name : Mukesh kumaar R.S Std. : Sec. :

Roll No. : 240701340 Sub :

S.No.	Date	Title	Page No.	Teacher's Sign / Remarks
1.	16.7.26	Study of various networks Command used in Linux & windows	1	
2.	23.7.26	Study of different types of network cables	5	
3.	30.7.26	CISCO Packet tracer Simulation	9	
4.		Setting up a LAN	11	
5.		Packet capture tool : wireshark	13	
6.		Python implementation of hamming code	15	
7.		Sliding window protocol	17	
8.		N-map host discovery	19	
9.		Subnetting	21	
10.a.		Internetworking with Routers	23	
10.b.		Internetworking using wireless Routers	25	
11.a.		Static Routing	27	
11.b.		ARP	29	
12.		End-end Configuration of Front laser	31	
13.		own ping program	35	
14.		Packet Sniffing	37	

Completed

Commands:

```
ipconfig/all          # windows
ip a                  # Linux
ping -c 4 8.8.8.8     # windows
traceroute google.com # Linux
netstat -taln         # Linux
nslookup google.com   # DNS Resolution
arp -a                # ARP Table
route -n              # Routing Table
```

ex no : 1 Basic Networking Commands in Linux and Windows

Aim:

To study and execute basic networking commands available in linux and windows operating system for diagonising and network issues.

Procedure:

1) open Command promt (windows) (or) Linux (Terminal)

2) execute "ip config" in windows (or) "ipconfig lipo" in Linux

3) use ping 8.8.8.8 to test connectivity to google DNS Server.

4) execute "tracert google.com" (windows) (or) "traceroute google.com (Linux) to view packet path.

5) Run "netstat -an" to list actions network connections and listing port

6) use "nslookup google.com" to perform DNS lookup

8) use "route print" to view routing table.

9) Record all outputs and analyse the results.

Result:

Basic Networking Commands were Successfully executed on both Linux and windows IP address, routing tables, ARP Caches and DNS resolution were observed and Recorded.

Commands :

Sudo ip addr add 192.168.1.10/24 dev eth0

Sudo ip route add default via 192.168.1.1

ip a show eth0

ping 192.168.1.1

EX
NO: 2 Learning and assigment of IP address Manually to Computers

Aim:

To understand IP addressing and manually assign a state IP address to Computers on a network.

Procedure.

1) Connect two (or) more Computers using a network Switch and Ethernet Cables.

2) Assign a state IP address, Subnet mask, gateway and DNS

3) example: IP 192.168.1.10, Subnetmask: 255.255.255.0, Gateway: 192.168.1.1

4) verify the IP Configuration using ip a (or) ipconfig

5) use Ping to check Connectivity between Computers.

output:

1) Static IP address was assigned Successfully

2) IP Configuration was verified using ipconfig

3) Ping reply received Successfully.

Result :

The Static IP address was successfully assigned and network Connectivity was verified using the "Ping" Command

Observation:

Straight - through: 568B - 568B

Cross - over : 568A - 568B

All 8pins Showed proper Continuity the Cable tester

Ex No: 3 Study of Network Cables and Crimping with RJ45 Connector

Aim:

To study n/w cables and prepared Straight-through and Cross-over Ethernet Cables using RS45 Connectors.

Procedure:

1) Cut the required length of UTP Cable and Strip the outer jacket

2) Arrange the wires in 568B order for the Straight-through Cable.

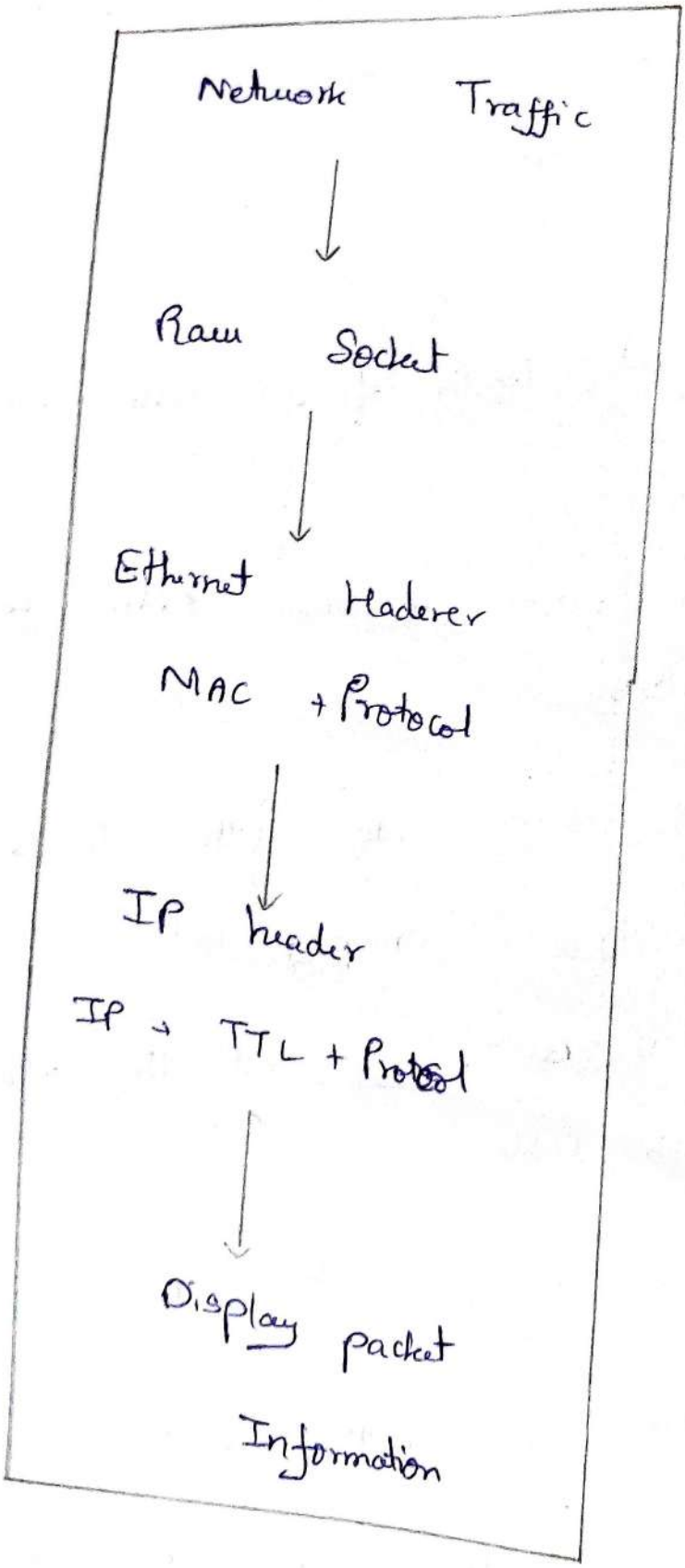
3) Insert the wires into the RJ45 Connector and Crimp using Crimping tool.

4) Repeat 568B wiring at the other end for Straight-through Cable.

5) For a Cross-over Cable. use 568P at one end & 568B at another end.

6) Test both Cables using a Cable tester & clide LED sequence.

Diagram :


Network Traffic
Raw Socket
Ethernet Haderer
MAC + Protocol
Ip header
Ip → TTL + Protocol
Display packet
Information

Result:

Straight and cross over ethernet cables were successfully prepared and tested using RJ45 Connectors.

3/8/2026

Ex: 4

Setting up a local area Network (LAN)

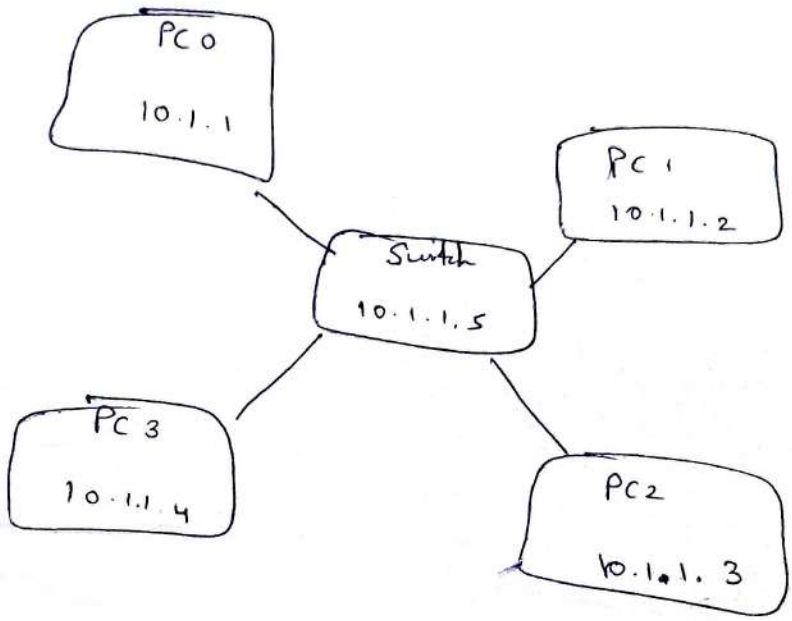
Aim:

To design, setup and configure a local area network (LAN) connecting few computers through a network switch and demonstrate resource/data sharing across the network.

Equipment Required:

Equipment	Qty	Specification
Personal Computer	4	Ethernet (NIC) enabled
network switch	1	8/16/24 port
Ethernet Cables (cat5e/cat6)	4	Straight through RJ45 Connectors

Network diagram



Implementation in CISCO packet tracer

1) Place the devices

2) Cable the devices

3) Assign IP address to PCs

4) Configure the Switch management IP

5) verify Connectivity using Ping

6) Demonstrate Data Sharing (Server simulation)

10.8.

Aim:

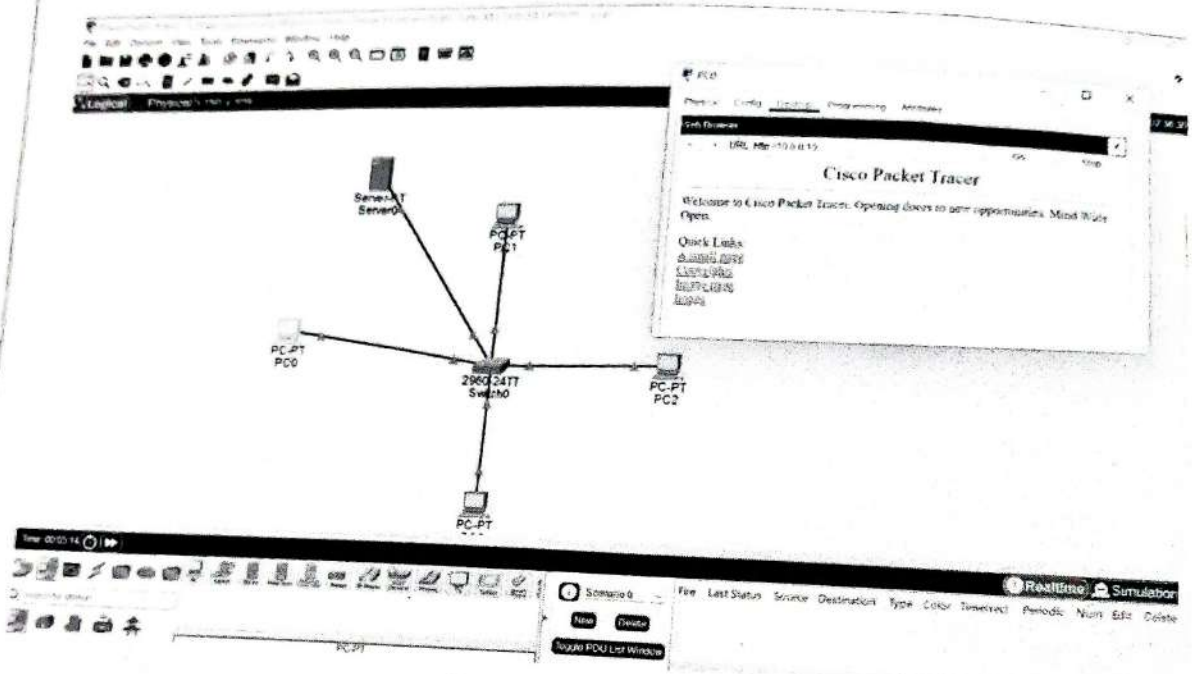
e

Packet

your

Protoc

Pack



Result:

Thus LAN is settled up and connection is verified successfully

Ex: 5

10.8.2026

experiment on Packet Capture tool wireshark

Aim:

experiments on Packet Capture tool wireshark

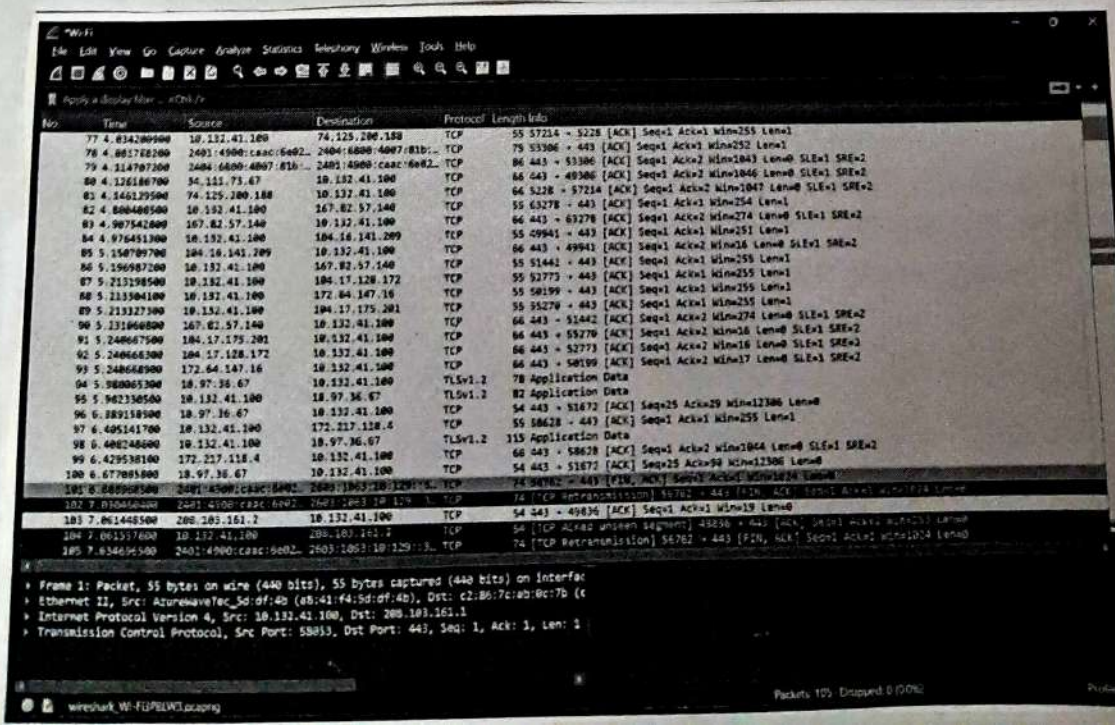
Packet Sniffer:

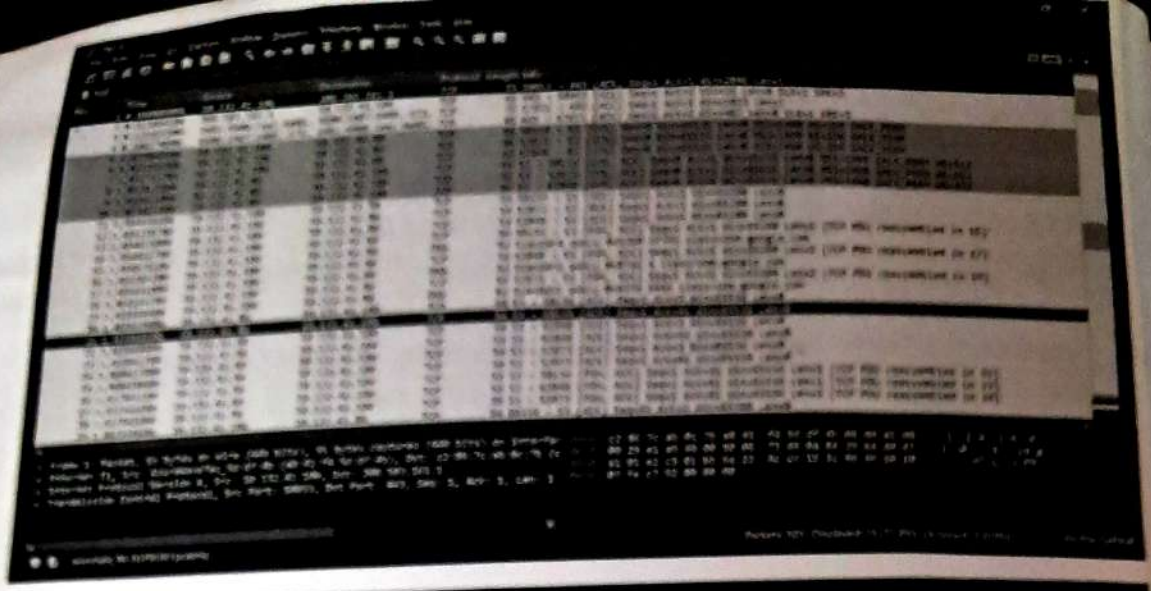
Sniffs messages being sent / received from/by your Computers

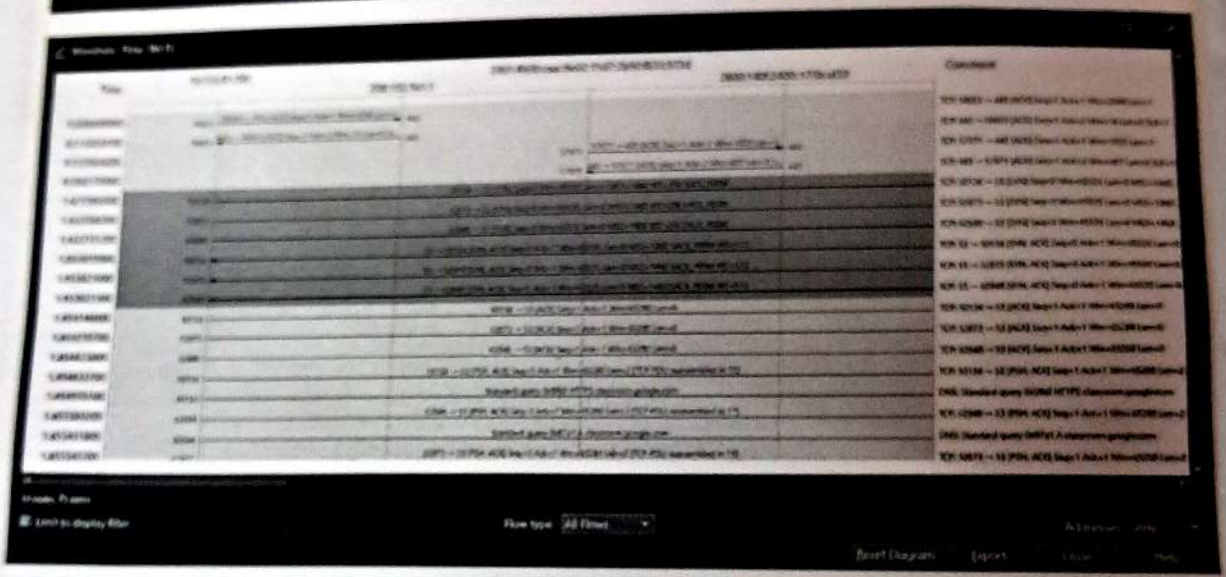
store and display the contents of the various Protocol fields in the messages

Packet Sniffer Structure diagnostic tool:

* Tepdump - eg - tupdump- enx host 10.123.4.2 -w exe3 out
* ~~Y~~wireshark - wireshark - r exe3 out

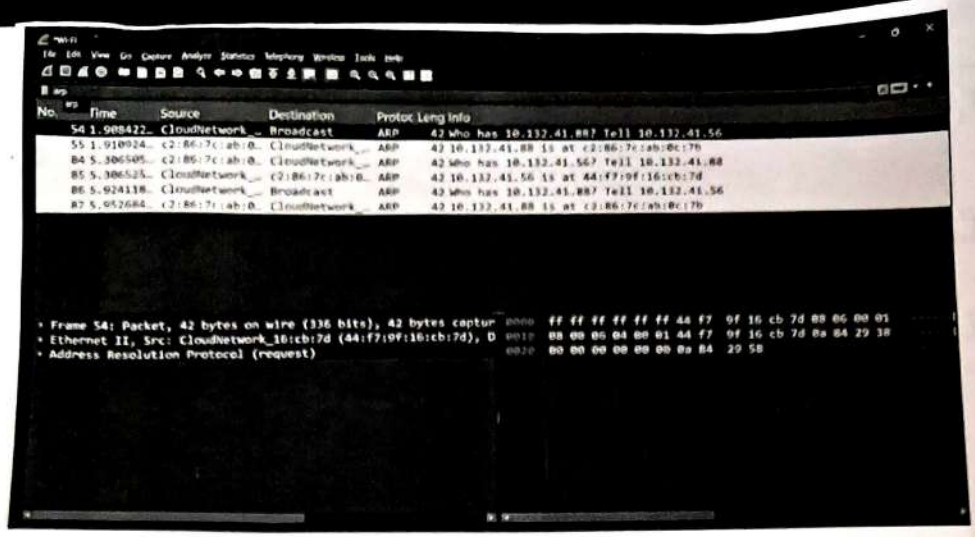


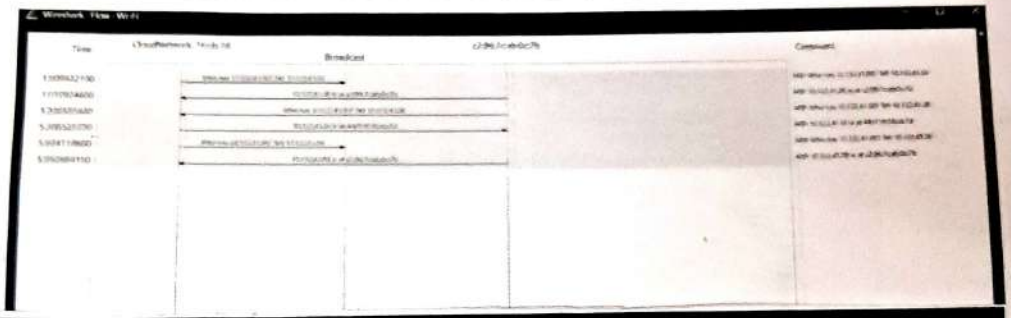


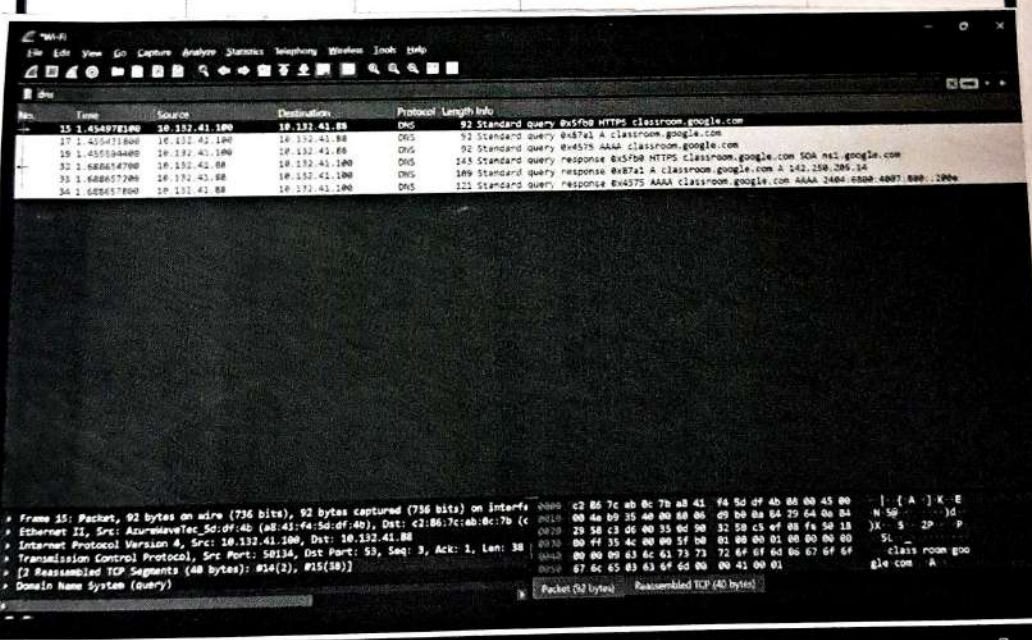


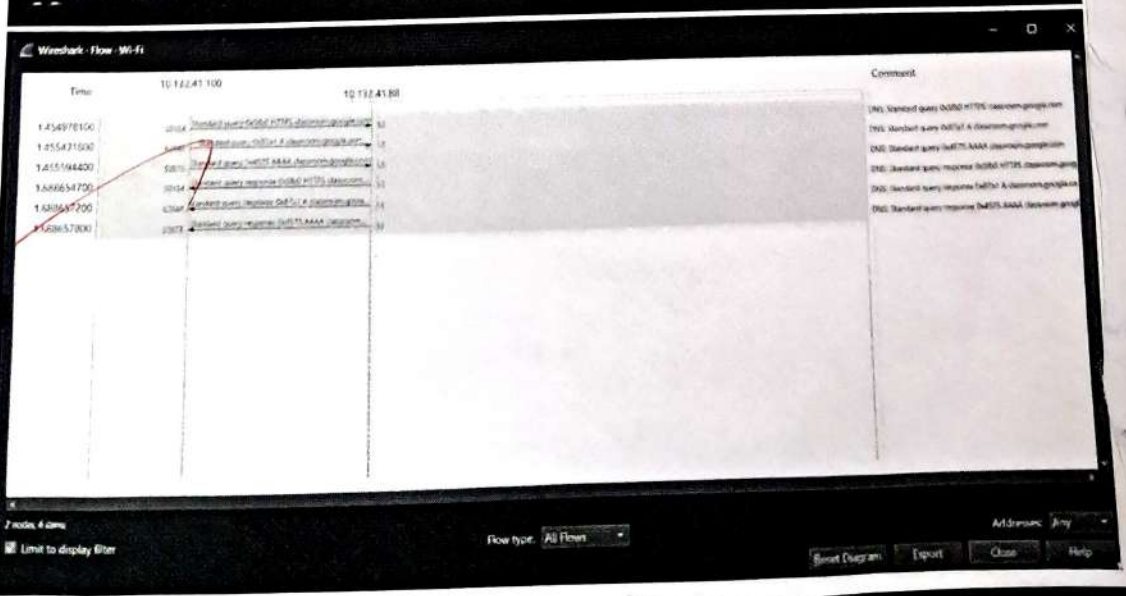
Pr

* Select the local area connection in wireshark
* Go to Capture option
* Select Capture and Stop automatically after 100 packets
* Then click Start Capture
* Save packets.


arp
No. Time Source Destination Protoc Leng Info
54 1.908422.. CloudNetwork_.. Broadcast ARP 42 Who has 10.132.41.88? Tell 10.132.41.56
55 1.910924.. c2:86:7c:ab:0.. CloudNetwork_.. ARP 42 10.132.41.88 is at c2:86:7c:ab:0c:7b
84 5.306505.. c2:86:7c:ab:0.. CloudNetwork_.. ARP 42 Who has 10.132.41.56? Tell 10.132.41.88
85 5.306525.. CloudNetwork_.. c2:86:7c:ab:0.. ARP 42 10.132.41.56 is at 44:f7:9f:16:cb:7d
86 5.924118.. CloudNetwork_.. Broadcast ARP 42 Who has 10.132.41.88? Tell 10.132.41.56
87 5.952684.. c2:86:7c:ab:0.. CloudNetwork_.. ARP 42 10.132.41.88 is at c2:86:7c:ab:0c:7b
Frame 54: Packet, 42 bytes on wire (336 bits), 42 bytes captur
Ethernet II, Src: CloudNetwork_16:cb:7d (44:f7:9f:16:cb:7d), D
Address Resolution Protocol (request)
0000 ff ff ff ff ff ff 44 f7 9f 16 cb 7d 08 06 00 01
0010 08 00 06 04 00 01 44 f7 9f 16 cb 7d 0a 84 29 38
0020 00 00 00 00 00 00 0a 84 29 58


Wireshark · Flow · Wi-Fi
Time
Broadcast
Comment


Wi-Fi
dns
No. Time Source Destination Protocol Length Info
15 1.454978100 10.132.41.100 10.132.41.88 DNS 92 Standard query 0x5fb8 HTTPS classroom.google.com
17 1.455471600 10.132.41.100 10.132.41.88 DNS 92 Standard query 0x87e1 A classroom.google.com
19 1.455594400 10.132.41.100 10.132.41.88 DNS 92 Standard query 0x4575 AAAA classroom.google.com
32 1.688654700 10.132.41.88 10.132.41.100 DNS 143 Standard query response 0x5fb8 HTTPS classroom.google.com SOA ns1.google.com
33 1.688657200 10.132.41.88 10.132.41.100 DNS 109 Standard query response 0x87e1 A classroom.google.com A 142.250.206.14
34 1.688657800 10.132.41.88 10.132.41.100 DNS 121 Standard query response 0x4575 AAAA classroom.google.com AAAA 2404:6800:4007:800::200e
Frame 15: Packet, 92 bytes on wire (736 bits), 92 bytes captured (736 bits) on interfa
Ethernet II, Src: AzureWaveTec_5d:df:4b (a8:41:f4:5d:df:4b), Dst: c2:86:7c:ab:0c:7b (c
Internet Protocol Version 4, Src: 10.132.41.100, Dst: 10.132.41.88
Transmission Control Protocol, Src Port: 50134, Dst Port: 53, Seq: 3, Ack: 1, Len: 38
[3 Reassembled TCP Segments (40 bytes): #14(2), #15(38)]
Domain Name System (query)
0000 c2 86 7c ab 0c 7b a8 41 f4 5d df 4b 08 00 45 00
0010 00 4a b9 35 40 00 80 06 d9 b0 0a 84 29 64 0a 84
0020 29 58 c3 d6 00 35 0d 90 32 58 c5 ef 08 fa 50 18
0030 00 ff 35 4c 00 00 5f b8 01 00 00 01 00 00 00 00
0040 00 00 09 63 6c 61 73 73 72 6f 6f 6d 06 67 6f 6f
0050 67 6c 65 03 63 6f 6d 00 00 41 00 01
Packet (92 bytes)
Reassembled TCP (40 bytes)


Wireshark · Flow · Wi-Fi
Time
10.132.41.100
10.132.41.88
Comment
Flow type: All Flows
Limit to display filter
Reset Diagram
Export
Close
Help

Ex:6

10/9/2026

i) Create a filter to display only TCP/UDP packets, inspect the packets and provide the flow graph

ii) Create a filter for ARP packets

* Go to Search option
* Search ARP packet in Search bar
* Save the packets.

iii) Creating a filter to display DNS packets and Provide the flow graph

iv) Creating a filter for ICMP and DHCP packets and inspect and show the packets

Result:

Wireshark helps us in analysing and understanding our network traffic and packet flow through various filters and flow graphs.

error

->Hammin

that

errors

to

Sender

Code

ex:6 Python Implementation of hamming Code Concept

10/9/2026

error correction at Data Link layer (ALHL)

->Hamming code is a set of error - correction codes that can be used to detect and correct the errors when the data is transmitted from the sender to the receiver.

Sender program:

-> Takes text input
-> Converts text to binary
-> Divides binary data into 4-bit groups
-> adds 3 redundant parity bits using hamming code
-> Saves the encoded data in channel text

Code:

```
def hamming-encode (data):
    encoded = ""
    for i in range (0, len(data), 4):
        d = data [i : i+4]
```

```
    if len(d) < 4:
        d = d.ljust(4, '0')
    d1, d2, d3, d4 = map(int, d)
    p1 = d1 ^ d2 ^ d4
    p2 = d1 ^ d3 ^ d4
    p4 = d2 ^ d3 ^ d4
    code = str(p1) + str(p2) + str(d1) + str(p4) + str(d2)
                                + str(d3) + str(d4)
    encode += code
return encoded
text = input("Enter text: ")
print("Data saved to channel.txt")
```

Receiver program:

Reads data from channel.txt

Checks every 7-bit hamming block

Finds the error position using syndrome

Code:

```
def hamming_code(data):
    corrected = ""
    error_found = False
    for i in range(0, len(data), 7):
        block = list(map(int, data[i:i+7]))
        if len(block) < 7:
            continue
        p1, p2, d1, p4, d2, d3, d4 = block
        s1 = p1 ^ d1 ^ d2 ^ d4
        s2 = p2 ^ d1 ^ d3 ^ d4
        s4 = p4 ^ d2 ^ d3 ^ d4
        error_position = s1 + (s2 * 2) + (s4 * 4)
Print ("\n corrected binary data:")
Print (binary_data)
if error_found:
    Print ("\n Error was detected and
                corrected")
else
    Print ("Received text:", text)
```

Ex: 7

24/09/2026

Aim:

Input:

Enter the text: HELLO

output:

Binary data: 01001 00001 00001 01001 00001 00011 11

Hamming encoded data: 10011 00111 00001 00010 00101 00100 01

Data Sent to channel. txt

Result:

Thus the hamming code was Successfully implemented to detect and Correct errors in the transmitted data.

Ex: 7
24/09/2026

Sliding window Protocol

Aim:

write a program to implement flow data control link layer using sliding window Protocol. Simulate the flow of frams from one node to another.

Sender Program:

```
import time
w = int (input ("window size : "))
msg = input ("message : ")
frams = [(i, c) for i, c in enumerate (msg)]
i=0
while i < len (frams):
    end = min (i+w, len (frams))
    with open ("sender - Buffer .txt", "w") as f:
    for n, c in frams [i: end]:
        Print ("frame", n, " : ", c)
        f. write (f" {n}, {c} \n")
```

Receiver program :

```
import time
import os
expected = 0
while true:
    if not os.path.exists("Sender_buffer.txt"):
        time.sleep(1)
        continue
    with open("Send_buffer.txt")as f:
        frame = f.readline()
    if not frame:
        time.sleep(1)
        Continue
    expected += 1
if not error:
    with open(Receiver_Buffer.txt', "w")as f:
        f.write("Ack:" + str(expected))
open("Sender_Buffer.txt", "w").close()
time.sleep(1)
```

Sender Code Input:

Window Size: 3

Message : HELLO

Sender Code output:

Frame 0: H

Frame 1: E

Frame 2: L

Frame 3: L

Frame 4: O

Receiver Code Input:

Sender Buffer.txt

0. H

1. E

2. L

3 L

4. O

Receiver Code output:

0. H

1. E

2. L

3. L

4. O

Result:

The Sliding window protocol was successfully implemented to control frame flow and handle acknowledgements and retransmissions.

Ex: 8 Nmap to discover the hosts using Nmap scans (ARP, TCMP, TCP/UDP) on the tryhackme platform

Aim:

To discover the hosts using Nmap scans (ARP, TCMP, TCP/UDP) on the Tryhacked Platform

Procedure;

1. Setup and environment Initialization:

Log in to the Tryhackme platform and navigate to the target room. Tryactive Nmap Room

display the target unreal machine and establish a VPN Connection using openVPN or launch the browser based attachbox

2. ARP Scan for local Host discovery:

Perform an ARP request Scan to identify Curiw Hosts on the local network segment

Icmp address mask Ping Scan:

Send ICMP address mask request packet to identify live systems by requesting subnet mask information.

TCP SYN Ping Scan:

Send TCP SYN Packet clinically to Port 80 (Common) Specified port) to discover system based on SYN - Ack (a) RST responses

Cmd:

Sudo nmap - PS - Sn <target - IP / Subnet>

TCP Ack ping Scan:

Send TCP Ack packet to client RST responses from achieve host, bypassing Stateless firewall

Crunches

Cmd:

Sudo nmap - PA - Sn <target - IP / Subnet>

UDP Ping scan

Send empty UDP packets to destination port to detect that via ICMP port unreachable response

Cmd:

Sudo nmap -PU -sn <target-IP/ subnet>

Disable Port Scanning (Host discovery only)

use the sn flg with nmap to Perform host discovery exclusively without triggering subsequence port scan

Cmd:

Sudo nmap -sn <target-IP / subnet>

Result:

Thus - live host discovery using nmap was successfully performed on the Try platform.

Aim:

To

network

IP

the

Procedure

Plan

Route

IP

ex: 9 Implementation of Subnetting in Cisco Packet Tracer Simulator

Aim:

To design, configure and Simulate a Subnetted network topology in cisco packet tracer design Static IP address using classes IP Subnetting across the Subnet with Cmd without a router

Procedure:

1. Design Topology: open cisco packet tracer and Place . Router, Switches and pcs on the workplace

2. Connect Cables: Connect the Switches and to Router interface using straight through Cables.

3. Assign IP addresses to Pcs Go to desktop IP Configuration each Pc and assign

Subnet mask: 235.255.255.224 (127)

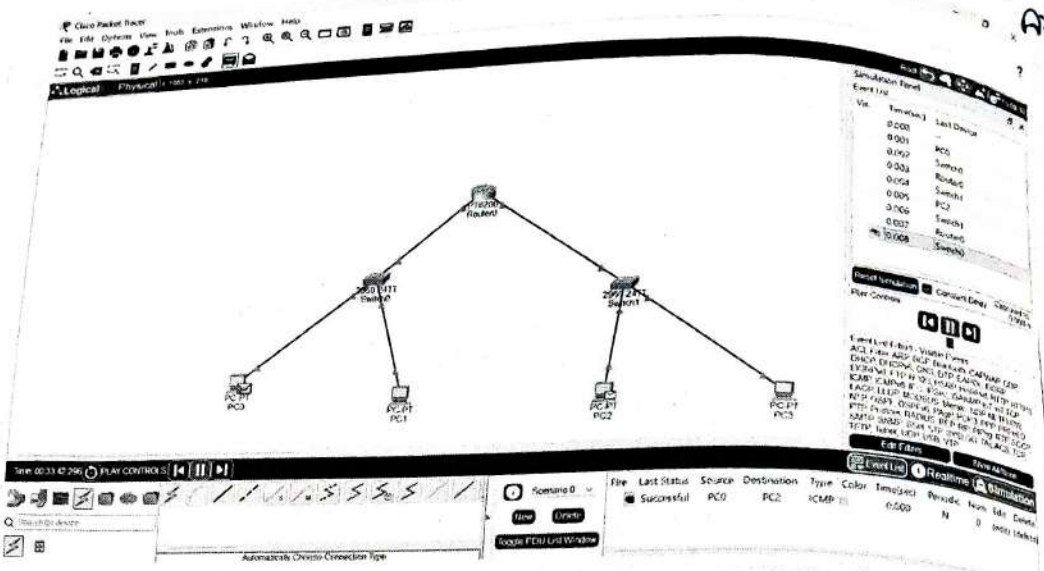
The and Gateways:

Subnet 1: Pc HS: 192.168.1.11 to 192.168.1.15 | Default gateway = 192.168.1.1

Subnet 2: Pc HS = 192.168.2.11 to 192.168.2.15 | default gateway = 192.168.2.1

4. Configure Router : open Router CLI

Configur interface.



5. Test Communication.

* with Router: Ping a Pc in Subnet 2 from Subnet 1 using Command prompt

* without Router: Connect the router and ping again. Ping will fail due to lack of inter. Sh routing.

Result!

The submitted network topology was successfully designed and configured in Cisco packet Tracer.

Ex: 10

Aim:

Inte

TRACER

Procedure

Step1

route

ente

S

Ex: 10 A) Interconnecting with routers

Aim:

Interconnecting with routers in CISCO PACKET TRACER Simulator.

Procedure

Step1 : Configuring Router

1) open cisco packet tracer & place one router & 2 Pc's

2) select the router & open the CLI. and press enter to access the command prompt

3) enter privileged mode using enable

4) enter global config

5) Configure fast ethernet 0/1 with

* IP address : 192.168.20.1
* Subnet mask : 255.255.255.0

Step2 : Configuring Pc's

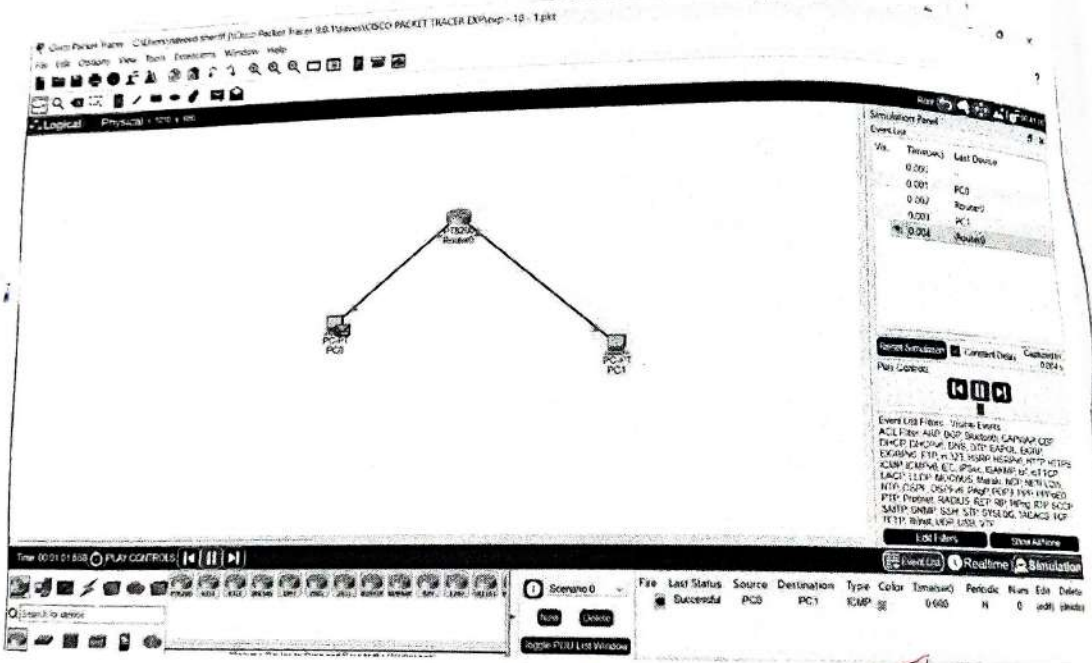
PC0 → 192.168.10.2

PC1 → 192.168.20.2

Step 3: Connecting A's with Router

* Connect PC0 ethernet 0 to router, fast ethernet 0/0 using a Copper Straight Cable

* Connect PC1 ethernet 0 to Router, Fast ethernet 0/1 using a Copper Straight Cable.



Result:

Thus a Single inter networking cross configured using a Router.

B) Inter net

Aim:

To design using a Internet

Procedure:

Step 1:

* open

like P

* Co

Step 2: Con

* u

->

*

*

8) Inter networking using wireless router

Aim:

To design & configure an inter network using a wireless router, DHCP server & Internet cloud in Cisco Packet tracer.

Procedure:

Step 1: Build the network topology

* open cisco packet tracer & add devices like PC, router & server
* Connect the devices using appropriate cable.

Step 2: Configure the router

* wireless router => GUI → wireless network
→ home network
* Go to setup tab & ensure DHCP is enabled
* Save the settings

Step 3

Configure the Pc

Step 4 Configure the Internet cloud

* Open the internet cloud
* Config → Cable
* Set: From port → Coaxial

To port → Ethernet

* Click add
* Go to ethernet * Select Cable as provider network

Steps: Configure the Cisco Com Server

DHCP Configuration

Paramter	value
Pool name	DHCP pool
Default Gateway	208.67.220.220
DNS Server	208.67.220.2
Subnet mask	255.255.255
Maximum user	50

Click add

DNS Config

Click

Step 6:

Resu

Internet

Click add

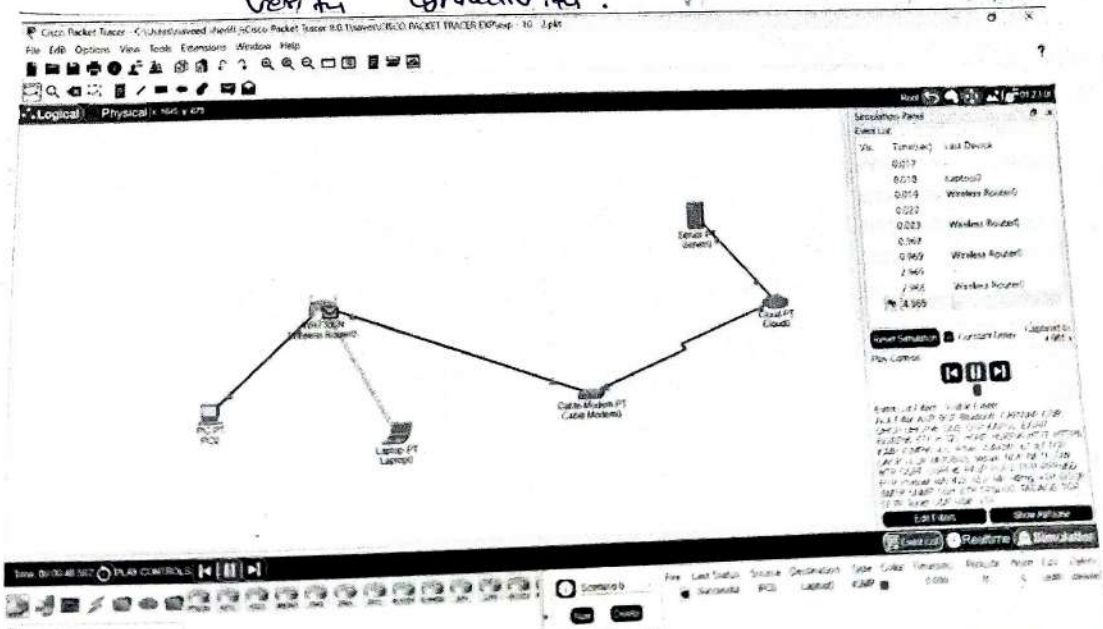
DNS Configuration

Paramder	value
Name	cisco.com
Type	A Record
addres	208.67.220.220

Click add

Step 6 :

Verify Connectivity.



Result

Internet cloud wireless router, other servers successfully configured.

B) RIP (Routing Information protocol)

Aim:

To simulate & Configure RIP using Cisco Packet tracer.

Procedure:

* open cisco packet tracer & create the topology.
* Assign IP addresses to Pc's & router interfaces
* Configure the fast ethernet & Serial interfaces of all routers
* use no shutdown to active
* For DCE Serial interfaces, Configure
* Configure RIP

router rip

network 10.0.0.0

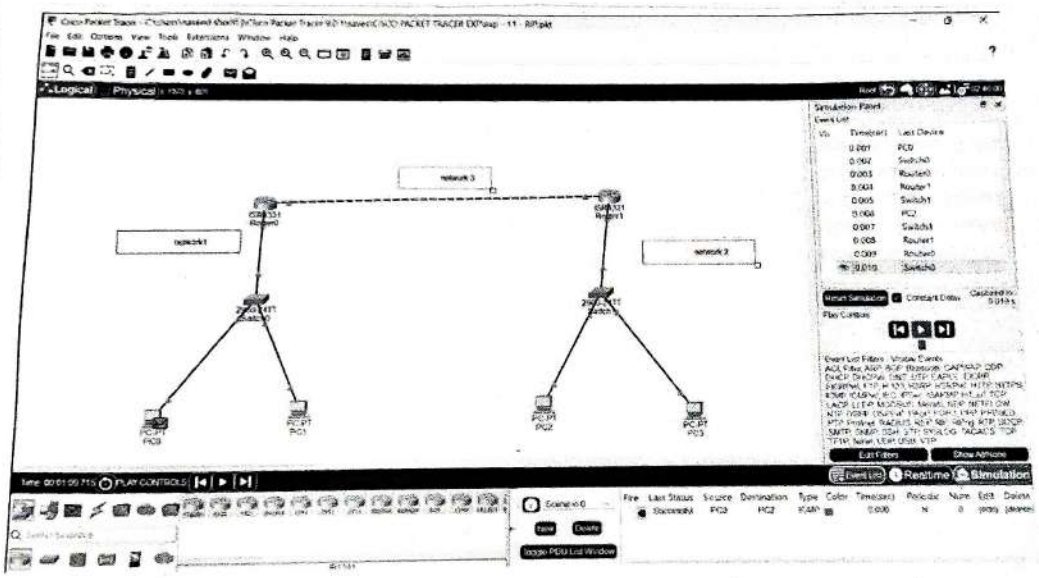
network 192.168.1.250

* Configure RIP on R1 using

router 192.168.1.244

router 192.168.1.248

* From PC0, Ping command to verify Connectivity with PC1.



Result:

Thus RIP was successfully configured & Simulated using Cisco packet tracer.

Ex : 12 End - end Communication at transport layer

a) Aim:

To implement echo client server using TCP / UDP sockets

Server:

```
import Socket
s = Socket . Socket()
s. bind (("127.0.0.1", 5000))
s. listen()
Print ("client", msg)
c. send (msg . encode())
c. close()
s. close()
```

Client:

```
import Socket
s = Socket . Socket()
s. Connect (("127.0.0.1", 5000))
msg =
s. send
Print (
s. cl
input
message : Hello
```

Result:

```
msg = input ("Enter message :")
S. send (msg. encode())
Print ("server", S. recv (1024). decode())
S. close()
```

input

Enter message : Hello server

output

Client connected

Received : hello server

Client echo from server : hello server.

Result:

Thus to implement echo client server using TCP/UDP sockets are implemented successfully.

b) Aim:

To implement chat client server using TCP / UDP Sockets.

Server:

```
import socket
s. bind (("127.0.0.1", 5001))
s. listen(1)
c, addr = s. accept()
Print ("client:", c. recv(1024) decode())
c. send (input ("server:") . encode())
c. close()
S. close()
```

Client:

```
import Socket
S= Socket. Socket()
S. Connect (("127.0.0.1", 5001))
S. Send (input ("client:") . encode())
Print ("server:", S. recv (1024). decode())
S. close()
```

Input:

Client : Hello

Server : Hi

Client : Bye

output:

Client : Hello

Server : Hi

Result:

This to implement chat client server using TCP/UDP sockets was completed successfully

Ex: 13

Ping program

Aim:

To implement our own ping Program

Code: my ping.py

```
import socket
import struct
import time

s = socket.socket(socket.AF_INET, socket.SOCK_RAW,
                  socket.IPPROTO_ICMP)

Packet = struct.pack("!BBHHH", 8, 0, 0, 1234, 1)

start = time.time()

Print("Reply received from:", addr[0])

Print("Time:", round((time.time() - start)*
      1000, 2), "ms")

s.close()
```

Linux / ubuntu => Sudo python3 my ping.py

Input:

Enter

output:

Reply

Rep

Recei

S

Input:

Enter host : 127.0.0.1

output:

Reply from 127.0.0.1 time = 1.23 ms

Reply from 127.0.0.1 time = 0.98 ms

Result:

The ping program was successfully Studied & implemented.

ex: 14

RAW Sockets

Aim:

To write a Code using RAW Sockets to implement packet Sniffing.

Code:

```
import Socket

s = Socket.Socket (Socket.AF_PACKET,
                   Socket.Sock_RAW,
                   Socket.htons(3))

Print ("Sniffer started. Press Ctrl+C to stop")

while True:

    Packet, addr = s.recvfrom(65535)

    dest = ":".join("%02x". Packet[0:6])

    src = ":".join("%02x" %b Packet[6:12])

    Print ("Source MAC:", src)

    Print ("Destination MAC:", dest)
```

Linux / ubuntu ⇒ Sudo Python3 Sniffer.py

Input: *

network

output:

Packet

Packet

Packet

Packet

Result:

The

Success

Capture

Input :

network packets

output :

Packet sniffer Started

Packet received from : 192.168.1.5

Packet received from : 192.168.1.1

Packet received from : 192.120.183.14

Result :

The RAW socket Sniffing Program was successfully implemented & network packet was captured & displayed.